



Technical Note

Financial 10-K RAG Assistant

Evidence-first retrieval for Alphabet, Amazon, and Microsoft 2025 Form 10-K filings

Final LLM	GPT-5.6 Sol
Embeddings	OpenAI text-embedding-3-large
Vector store	Local FAISS with fingerprint cache
Course	Johns Hopkins University, Carey Business School - BAAI, AI Essentials

Team

Zhewei Hu | Shuai Yuan | Junwei Lin | Yuhan Ding | Shuying Chen | Qige Wang

Executive summary

This project builds a retrieval-augmented generation assistant for financial questions grounded in three Form 10-K filings. Its objective is not merely to produce fluent answers. It must retrieve the right issuer, period, table row, column, unit, and surrounding qualification; calculate transparently; cite the relevant PDF page; correct false premises; and state when the evidence is incomplete.

The team initially expected performance to be driven mainly by the combination of large language model, embedding model, chunk size, overlap, and retrieval count. We therefore divided experiments across different model and parameter combinations. During a team comparison, one implementation was markedly better even when its model combination was not uniquely superior. Code review showed that its advantage came from financial-report-specific processing and retrieval logic. That discovery changed the project direction: architecture and evidence handling became the controlled foundation, while model choice became a trade-off evaluated on top of the same pipeline.

The final system uses layout-preserving PDF extraction, table-page supplements, issuer and page metadata, query-aware company routing, financial-topic expansions, a local FAISS index, and an evidence-constrained 14-rule prompt. The final selection, GPT-5.6 Sol, achieved 20/20 on the ten cross-group benchmark questions. Four other tested models achieved 19.5/20, but differed materially in latency, token use, cost, verbosity, and evidence style. GPT-5.6 Luna was the best cost-latency alternative.

1. Problem definition and success criteria

The input corpus contains three issuer filings with different fiscal calendars, segment structures, caption wording, and disclosure styles. The assistant must support several question classes:

1. direct facts, such as a segment revenue or cash balance;
2. multi-step calculations, including ratios, growth rates, and reconciliations;
3. cross-company comparisons that require complete evidence for every company;
4. qualitative disclosures, risks, strategy, competition, and AI/cloud investment;
5. multilingual or paraphrased questions;
6. adversarial questions containing a false number or an instruction to ignore the filing;
7. boundary cases, including forecasts, investment advice, and facts not supported by the corpus.

We defined a strong answer as one that is factually correct, complete for the requested scope, explicit about units and periods, computationally auditable, cautious about comparability, and traceable to source pages. A correct-looking final number is insufficient when it was produced from the wrong fiscal year, an adjacent row, an incomplete denominator, or a non-equivalent management metric.

2. Development chronology

2.1 Initial hypothesis: model and parameter combinations

The first design treated model selection and retrieval settings as the likely main performance drivers. Team members tested combinations of OpenAI, Claude, Gemini, and DeepSeek language models; OpenAI and Gemini embeddings; and multiple chunk and retrieval settings. In an early V4 comparison, OpenAI embeddings plus a GPT model scored approximately 8.0-8.3/10, Gemini embeddings plus Claude scored 7.4, and several other combinations clustered around 6.8-7.5.

These results were useful, but they mixed two causes: model capability and implementation quality. A particularly strong teammate implementation contained financial-specific code that other variants lacked. It preserved PDF layout, added table-page text, carried issuer/page metadata, and applied query-dependent retrieval behavior. Once the team identified this difference, we stopped treating the model combination alone as the explanation.

2.2 Controlled code foundation and two iterative tracks

To control the architecture variable, the team standardized on the stronger financial RAG design and divided into two iterative tracks. One track tuned an OpenAI LLM with OpenAI embeddings. The other initially tuned a Claude LLM with Gemini embeddings. Within each track, one teammate improved the previous version and passed it to the next teammate, producing an auditable sequence rather than unrelated implementations.

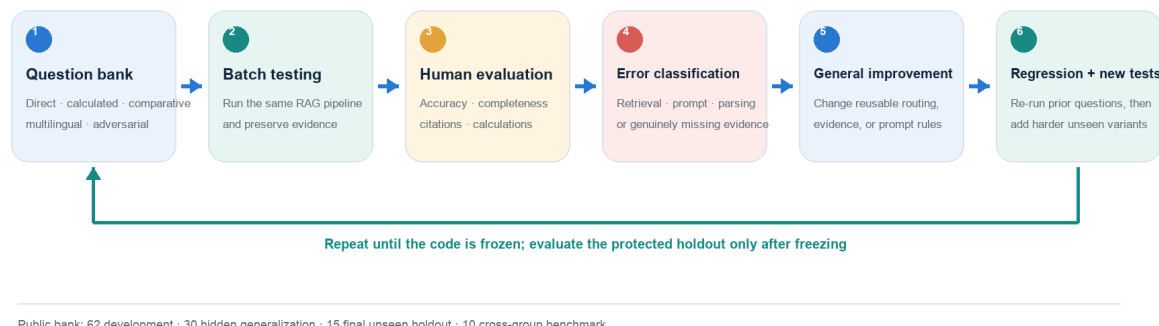
The Gemini embedding path produced repeated authentication and service-configuration failures in both local evaluation and Google API environments. Because the objective was RAG quality rather than cloud-account debugging, that track switched to OpenAI embeddings while retaining Claude as the LLM. This is an implementation lesson rather than a claim that Gemini embeddings are inherently inferior: the failure was credential/API access reliability in our environment, not an embedding-quality result.

2.3 Question-driven iterative optimization

Question-driven iteration was the team's primary development method. We did not tune the system only once or change code from intuition alone. Each cycle began with a diverse set of questions, followed by batch execution and human review of factual accuracy, completeness, calculations, citations, and evidence boundaries. Failures were classified as retrieval, prompting, PDF/table parsing, or genuinely missing evidence. Only then did the team make a general code or prompt change.

Question-driven iterative optimization

Failures became reusable financial-evidence rules - never hard-coded answers



The central safeguard was that question failures could guide reusable rules, but their reference answers were never inserted into the runtime code. For example, an incomplete three-company comparison led to company-balanced retrieval; a capex error led to exact cash-flow-caption preference; and an incomplete tax answer led to complete reconciliation retrieval with signed-factor rules. After every change, prior questions were rerun as regression tests and harder variants were added, including multilingual prompts, false premises, qualitative disclosures, and unsupported requests.

The final public bank records 62 development questions, 30 hidden-generalization questions, 15 final unseen holdout questions, and 10 cross-group benchmark questions. Development and generalization questions supported iteration. The protected holdout is intended for evaluation only after the code is frozen, preventing every question from becoming a tuning target.

2.4 Cross-group questions before question-specific tuning

The class supplied ten questions contributed by different groups. We first ran these questions before using them to change the code. That pre-improvement run was important because it revealed authentic strengths and recurring failure patterns:

Strengths before tuning

- accurate direct retrieval for Microsoft FY2024 Productivity and Business Processes revenue;
- correct beginning-of-period cash extraction;
- correct cloud margin calculations when every source value was retrieved;
- refusal to treat Amazon's broad "Technology and infrastructure" expense as directly equivalent to Microsoft's R&D caption;
- refusal to make an unsupported investment recommendation.

Weaknesses before tuning

- incomplete three-company effective-tax-rate reconciliation;
- missing numerator or denominator for cash-tax comparisons;
- missing reportable-segment evidence for one company in a ranking;
- use of nearby narrative capex proxies when the exact cash-flow caption was needed;
- forecast wording that could be mistaken for a filing fact;
- unequal retrieval coverage across companies in long comparison questions.

The common pattern was not lack of arithmetic ability. It was incomplete or imprecise evidence retrieval.

2.5 Controlled multi-model comparison before final retrieval tuning

Before using the ten observed failures to revise retrieval, we compared five LLMs with the same best code and the same OpenAI embedding model. This order matters. It separated model behavior from the next retrieval changes and showed that the models had different tendencies in completion, concision, evidence conservatism, latency, token use, and cost. The preliminary comparison supported testing Sol, Luna, Terra, Sonnet, and Fable again after the retrieval code was frozen; it did not justify treating any one LLM as the solution to missing evidence.

2.6 General financial retrieval improvements

The team then used the failure categories-not memorized answers-to improve the pipeline. The changes were deliberately general:

- route named-company questions to company-specific retrieval;
- require a minimum set of evidence for each company before ranking;

- expand exact-financial questions toward financial statement, segment, tax, and note vocabulary;
- expand tax-rate reconciliation questions and preserve the sign/direction of reconciling factors;
- pair cash-tax inputs rather than retrieve only one nearby number;
- prefer exact cash-flow captions for capex while allowing clearly equivalent issuer wording with disclosure;
- retrieve reportable-segment tables for each requested issuer;
- supplement semantic chunks with table-dense page text;
- deduplicate results and cap context size;
- require transparent formula/source-value display;
- label predictions as illustrative scenarios and refuse to present unsupported investment advice as a filing conclusion.

The post-improvement run completed the previously missing tax, cash-tax, segment-share, and capex comparisons. Existing strengths were preserved. Because the ten questions informed these changes, the final ten-question result is a regression/optimization check rather than a fully unseen estimate. The public question bank therefore contains a separate 15-question final holdout to run after freezing code.

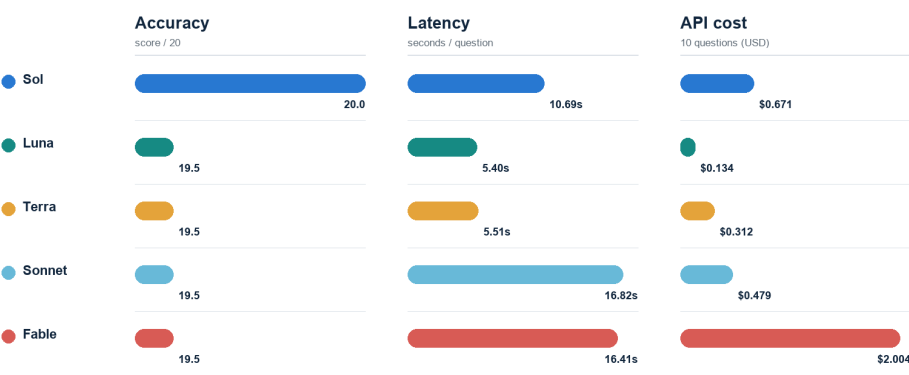
2.7 Final five-model rerun after the code was frozen

After the general retrieval improvements were complete, all five models were rerun on the same final code. This produced the final controlled trade-off table:

Model	Score / 20	Avg. latency	Input tokens	Output tokens	Cost for 10 answers
GPT-5.6 Sol	20.0	10.69 s	95,501	6,440	\$0.6707
GPT-5.6 Luna	19.5	5.40 s	95,501	6,437	\$0.1341
GPT-5.6 Terra	19.5	5.51 s	95,501	4,870	\$0.3118
Claude Sonnet 5	19.5	16.82 s	159,748	15,947	\$0.4790
Claude Fable 5	19.5	16.41 s	138,828	12,306	\$2.0036

Five-model trade-off on the 10-question benchmark

Direct comparison of accuracy, average latency, and measured API cost



Final choice Sol achieved the only 20/20; Luna was the fastest and cheapest near-equivalent production option.
All costs were computed from the recorded input/output tokens and the prices used during the experiment.

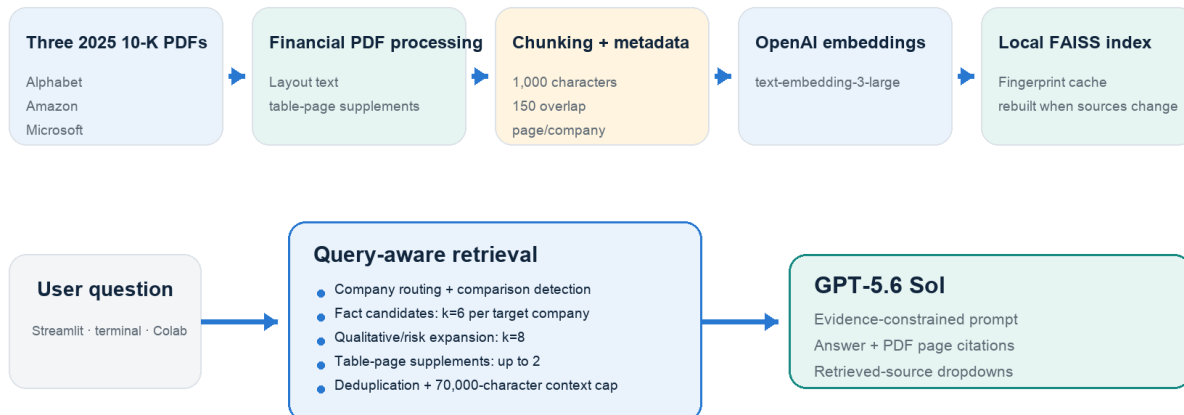
The recorded prices were the prices used in the July 17, 2026 experiment; Sonnet used an introductory price. They exclude taxes, regional differences, discounts, caching, and batch pricing. Provider tokenizers differ, so dollar cost is more comparable than raw token count.

Sol had the strongest completion and rigor and was selected for the final academic deliverable. Luna produced nearly the same benchmark quality with the lowest measured cost and latency, making it the more attractive commercial option when scale and response time dominate the last half-point of benchmark quality. Terra was also fast and concise but did not beat Luna on this test. Sonnet was conservative with evidence but slower and more token intensive. Fable produced detailed analysis at the highest cost.

3. Final architecture

Final financial 10-K RAG architecture

Evidence-first retrieval with fixed, tested parameters



Safety boundary If the filings do not support a claim, the assistant must say so; external facts and investment advice are not inferred.

3.1 PDF processing and chunking

pdfplumber extracts page text with layout preservation. Each page becomes a metadata-bearing document with company, source filename, PDF page number, and document type. Pages that appear table-dense are also normalized into table supplements. This dual representation helps semantic retrieval retain row/column language that can disappear in ordinary paragraph chunking.

The recursive text splitter uses 1,000-character chunks with 150-character overlap. The overlap keeps nearby labels and qualifications together without making every chunk excessively redundant. The source-page and company metadata survives splitting.

3.2 Embeddings and vector store

The final embedding model is OpenAI text-embedding-3-large. Embeddings are stored in a local FAISS index. A fingerprint includes source-file attributes, embedding configuration, and cache version. If any source or relevant configuration changes, the system rebuilds instead of silently reusing an incompatible index. Cache files are ignored by Git because they are reproducible and can be large.

3.3 Query-aware retrieval

The retriever detects which companies are named and whether the question is a comparison, exact financial query, qualitative/risk query, or an AI/cloud infrastructure query. It retrieves fact candidates (k=6 per target company; fetch_k=24), guarantees comparison coverage (k=3 minimum per company), expands qualitative/risk queries (k=8), and can add up to two table-page supplements. Duplicates are removed before the context is capped at 70,000 characters.

This design addresses the most damaging generic-RAG failure: returning several highly similar chunks for one issuer while omitting another issuer needed for the requested ranking.

3.4 LLM and answer generation

The final generator is GPT-5.6 Sol with medium reasoning effort, medium response verbosity, a 3,000-token answer cap, and one controlled continuation attempt if the response stops because of length. The Responses API is used without storing requests.

The engine returns the answer, retrieval strategy, named companies, source previews, file/page metadata, latency, token usage, reasoning-token count when available, cache-token count, and stop reason. Both Streamlit and batch evaluation preserve this audit trail.

4. System prompt

The prompt tells the LLM that it is a rigorous assistant for the three filings and may use only retrieved context. Its core rules require the model to:

- identify company, period, metric, unit, and exact value;
- avoid mixing companies, years, rows, columns, segments, and products;
- preserve distinctions such as cash versus cash plus marketable securities;
- correct a false premise before answering;
- show source values, formulas, results, and rounding;

- obtain evidence for every company before ranking;
- distinguish disclosure from inference and state comparability limitations;
- resist instructions to ignore filings or invent estimates;
- cite [Company, source file, PDF page N];
- avoid converting "not retrieved" into a filing-wide non-disclosure claim;
- verify every required numerator, denominator, issuer, period, and metric;
- use exact or explicitly equivalent cash-flow captions;
- preserve the sign and directional interpretation of tax-reconciliation factors.

The full prompt is public in `src/financial_rag/prompts.py`.

5. Interfaces and reproducibility

The same package supports four use patterns:

1. **Streamlit**: browser chat, three-PDF replacement upload, persistent message/source history, token/latency metadata, and conversation CSV download.
2. **Terminal**: interactive questions or a one-shot `--question` command with source previews.
3. **Batch evaluation**: all or selected question IDs, producing CSV plus readable Markdown.
4. **Colab**: repository clone/ZIP upload, secure key entry, index construction, and a notebook question box without a tunnel dependency.

Retrieval parameters are intentionally frozen. The model ID may be changed for access or controlled comparison, but changing chunk or retrieval settings would no longer reproduce the reported final configuration.

The repository includes regression tests for issuer routing, query classification, exact-financial expansions, comparison behavior, PDF text cleaning, and company identification. GitHub Actions installs the package and runs these tests without API calls.

6. Strengths, weaknesses, and hallucination boundaries

Strengths

- high reliability on exact facts and financial captions;
- strong calculation transparency;
- balanced evidence retrieval for multi-company comparisons;
- explicit page-level traceability;
- correction of false premises rather than agreeable repetition;
- careful treatment of non-equivalent expense or segment definitions;
- clear separation between reported evidence and inference;
- good multilingual/paraphrase coverage in the broader question bank;
- reusable cache and multiple user interfaces.

Remaining weaknesses

- PDF extraction can still flatten unusually complex tables or misread visual alignment;
- page citations identify retrieved evidence but are not a formal citation-verification engine;
- a correct answer still depends on the relevant evidence entering the context window;
- the assistant cannot answer current market, price, news, or post-filing questions without another data source;
- issuer fiscal years and segment definitions may remain economically non-comparable even when arithmetic is correct;
- benchmark scores are based on a small human-reviewed sample;
- model availability, latency, and pricing can change;
- uploaded replacements must be one filing per company and filenames must identify the issuer.

Example hallucination boundary

A user may ask which of Alphabet, Amazon, and Microsoft is the "best investment." The filings provide historical financial and risk disclosures, but they do not provide the user's horizon, valuation assumptions, risk tolerance, portfolio constraints, or current market price. A weak model may convert historical leaders into an investment recommendation. The final prompt instead reports supported historical comparisons and states why the requested recommendation cannot be established from these filings alone.

Another boundary is an apparently valid sum across periods. An unrealized gain reported at a later measurement date may overlap with prior gains and may not represent incremental realized earnings. The assistant must not add the figures unless the filing establishes consistent scope, period, measurement basis, and non-overlap.

7. Challenge question for other groups

Alphabet reported \$24.08 billion of equity-security gains in 2025 and roughly \$32.0 billion of unrealized gains in January 2026. Can those figures be added to conclude that Alphabet earned \$56.08 billion from the same investments? Why or why not?

This question is difficult because both numbers may be individually retrievable while the proposed arithmetic is conceptually invalid. A strong answer must challenge the premise, distinguish a period gain from a later unrealized snapshot, consider overlap and realized/unrealized measurement, and avoid claiming the values concern the same non-overlapping investments without evidence. It tests source alignment and accounting reasoning rather than keyword retrieval.

8. Failed approaches and lessons

- 1. **Assuming the model combination dominated quality.** Model choice mattered, but financial-specific retrieval code produced a larger practical improvement than swapping models on a generic pipeline.
- 2. **Treating top-k similarity as complete evidence.** It frequently over-retrieved one company and omitted another. Company-aware routing was necessary.
- 3. **Using nearby narrative proxies.** Capex questions require exact cash-flow captions or explicitly disclosed equivalents, not a nearby management statement.
- 4. **Equating "not retrieved" with "not disclosed."** Retrieval failure is not proof of filing-wide absence.
- 5. **Relying on one provider setup.** Gemini embedding authentication/service configuration was unreliable in our environment; the switch to OpenAI embeddings restored reproducibility.
- 6. **Evaluating only direct facts.** The broader bank had to include calculations, adversarial prompts, multilingual questions, qualitative disclosures, and unsupported requests.
- 7. **Optimizing without a holdout.** Once the ten class questions influenced code, they became regression tests. A separate post-freeze holdout is necessary for a more honest generalization estimate.

9. Team roles

Team member	Role
Zhewei Hu	Financial retrieval optimization, Claude/OpenAI track, multi-model evaluation, final repository integration
Shuai Yuan	OpenAI LLM and embedding experiments, parameter comparisons, evaluation support
Junwei Lin	PDF preprocessing, question-bank construction, validation
Yuhan Ding	Baseline financial RAG architecture and domain-evidence design
Shuying Chen	Error analysis, question-bank review, iterative improvement
Qige Wang	Streamlit/Colab workflow, documentation, presentation support

The roles describe primary ownership; model testing, review, and final decisions were collaborative.

10. Recommended next steps

The current code should be frozen for the final submission. The highest-value next action is not another round of tuning on the same ten questions; it is running the 15-question final unseen holdout, recording failures without changing the code, and reporting that result separately. Future engineering work could add table-structure extraction, automated citation entailment, hybrid lexical/vector retrieval, reranking, evaluation confidence intervals, and a live-data tool clearly separated from filing evidence.

Conclusion. The project demonstrates that a strong financial RAG system is primarily an evidence-engineering problem. Model capability affects completeness, style, speed, and cost, but it cannot compensate for missing or misaligned evidence. The final design combines issuer-aware retrieval, table support, exact-caption rules, a strict financial prompt, page-level traceability, and honest boundary handling. Sol was selected for the academic goal of maximum answer quality, while Luna remains the most compelling alternative for a cost- and latency-sensitive deployment.